

FACULTAD DE INGENIERÍA



TÍTULO DEL PROYECTO:

Sistema de Administración para una Ferretería

INTEGRANTES:

Steven Andres Guerrero Zuñiga 1052386584

Samuel Felipe Casallas Palomo 1014268700

Curso

Programación Orientada a Objetos

Docente

Rafael Antonio Acosta Rodriguez

Bogotá-Colombia

27 de julio de 2026

ÍNDICE

1. Introducción	1
1.1 Descripción del proyecto	1
1.2 Objetivo general	1
1.3 Objetivos específicos	1
1.4 Exclusiones	1
CAPÍTULO I. PROGRAMACIÓN ORIENTADA A OBJETOS	2
1.1 Arquitectura del proyecto	2
1.2 Diagrama de clases (UML)	3
1.3 Diagrama de casos de uso	3
1.4 Descripción de las clases principales	4
1.5 Aplicación de la Programación Orientada a Objetos	6
1.5.1 Abstracción	7
1.5.2 Encapsulamiento	7
1.5.3 Herencia	9
1.5.4 Polimorfismo	9
1.5.5 Interfaces	16
CAPÍTULO II. INTERFAZ GRÁFICA	18
2.1 Interfaz de usuario	18
2.2 Pantalla de inicio	19
2.3 Módulos principales	21
CAPÍTULO III. BASE DE DATOS	34
3.1 Diseño de la base de datos	34
3.2 Tablas de la base de datos	34
3.3 Modelo relacional	38
CAPÍTULO IV. RESULTADOS	38
4.1 Resultados obtenidos	39
4.2 Problemas encontrados durante el desarrollo.....	40
Conclusiones	40
Bibliografía	41

1.Introducción

1.1 Descripción del proyecto

Propósito del Programa

El propósito del sistema es desarrollar un sistema de escritorio para administrar una ferretería. Con esta solución se busca reducir errores en el manejo de la información, optimizar los procesos administrativos de la ferretería, mantener la integridad de los datos almacenados y facilitar el acceso rápido a la información necesaria para el desarrollo de las operaciones comerciales.

1.2 Objetivo general

Desarrollar una aplicación de escritorio para la administración de una ferretería que permita gestionar clientes, productos, inventario, categorías, ventas y usuarios, aplicando los principios de la Programación Orientada a Objetos y utilizando una base de datos MySQL para el almacenamiento de la información.

1.3 Objetivos específicos

Objetivos Específicos

- **Gestión de Clientes y Proveedores:** Implementar módulos funcionales para registrar, buscar, actualizar y eliminar la información de clientes y proveedores.
- **Control de Inventarios (Productos):** Automatizar la administración del catálogo de productos, permitiendo la actualización en tiempo real de existencias, precios y códigos de identificación almacenados en la base de datos.
- **Procesamiento y Gestión de Ventas:** Desarrollar un módulo de punto de venta que permita generar órdenes de pedido locales o con envío a domicilio, calcular subtotales y aplicar reglas de negocio para el cálculo de totales y descuentos.
- **Seguridad y Control de Acceso:** Diseñar un mecanismo de autenticación de usuarios (Login) con validación de credenciales para restringir el acceso no autorizado a los módulos de administración del sistema.
- **Generación de Documentos y Reportes:** Proveer una estructura modular que permita la emisión e impresión de facturas en diferentes formatos (ticket o factura de oficina) según el tipo de venta realizada.

1.4 Exclusiones

- El sistema no contará con un sistema contable
- No conectado a un servicio en línea

Análisis del sistema

- Lenguaje de programación: Java 11 o superior
- FrameWorks: JDBC para la conexión a la base de datos, Java Swing para la interfaz gráfica.
- Base de datos: MySQL (v8.0) o cualquiera que sea compatible

2.1 Requisitos Hardware

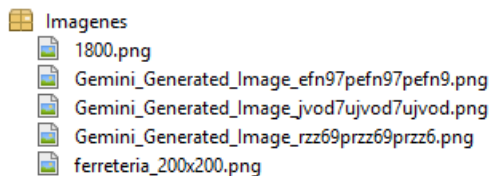
- Procesador de 2.0 GHz o superior
- Mínimo 4 GB de RAM
- Espacio en disco duro: 200 MB para la aplicación y datos locales

CAPÍTULO I. PROGRAMACIÓN ORIENTADA A OBJETOS

1.1 Arquitectura del proyecto

La estructura general del proyecto se divide en los siguientes componentes:

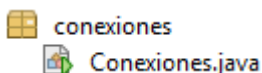
Paquete Imagenes: Almacena los recursos gráficos utilizados por la aplicación, como logotipos, iconos y fondos empleados en las diferentes ventanas del sistema.



Paquete Ventanas: Contiene todas las interfaces gráficas del sistema, incluyendo el inicio de sesión y los módulos para la administración de clientes, productos, categorías, ventas, inventario y usuarios.



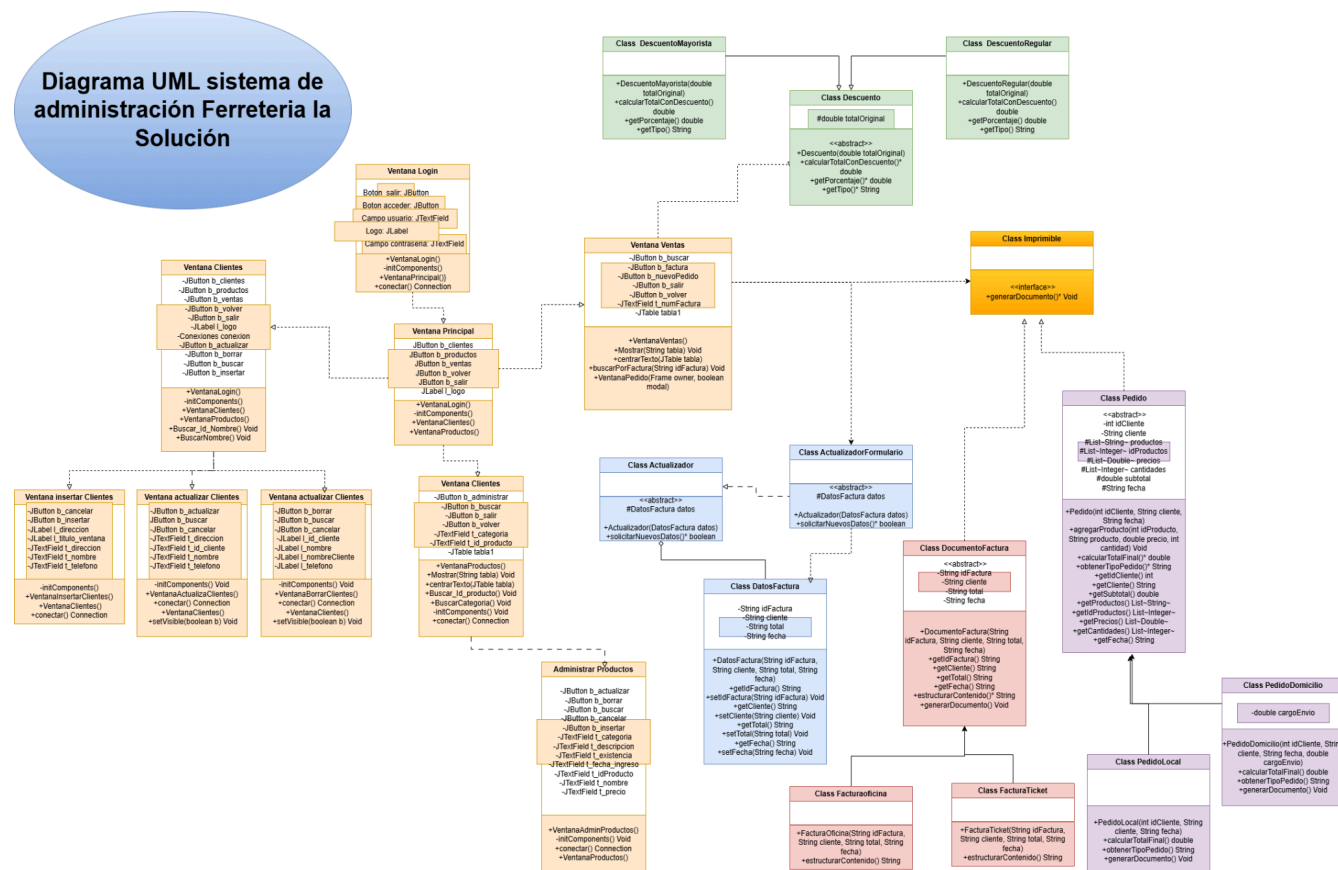
Paquete conexiones: Contiene la clase Conexiones, responsable de establecer la comunicación con la base de datos MySQL mediante JDBC.



Base de datos MySQL: Almacena la información correspondiente a clientes, productos, categorías, ventas, inventario y usuarios del sistema.

- Actualizador.java
- ActualizadorFormulario.java
- DatosFactura.java
- Descuento.java
- DescuentoMayorista.java
- DescuentoRegular.java
- DocumentoFactura.java
- FacturaOficina.java
- FacturaTicket.java
- Imprimible.java
- Pedido.java
- PedidoDomicilio.java
- PedidoLocal.java

Diagrama UML sistema de administración Ferreteria la Solución



1.3 Diagrama de casos de uso

El empleado será el encargado de interactuar directamente con los clientes y gestionar las operaciones del sistema. Entre sus principales funciones se encuentra la atención tanto a personas naturales como jurídicas, registrando sus pedidos y generando la factura correspondiente según cada situación.

Además, el empleado será responsable de la administración de la información relacionada con clientes, productos y ventas, garantizando que los datos se mantengan actualizados y organizados. También verificará la disponibilidad de existencias en el almacén antes de realizar una venta, con el fin de asegurar que los productos solicitados se encuentren en stock.

Dentro de sus actividades administrativas, podrá registrar nuevos clientes, agregar nuevos productos al inventario, actualizar información existente y consultar el historial de ventas cuando sea necesario. De esta manera, el empleado actúa como el principal usuario del sistema, asegurando el correcto funcionamiento de los procesos comerciales y el control eficiente de los recursos de la empresa.

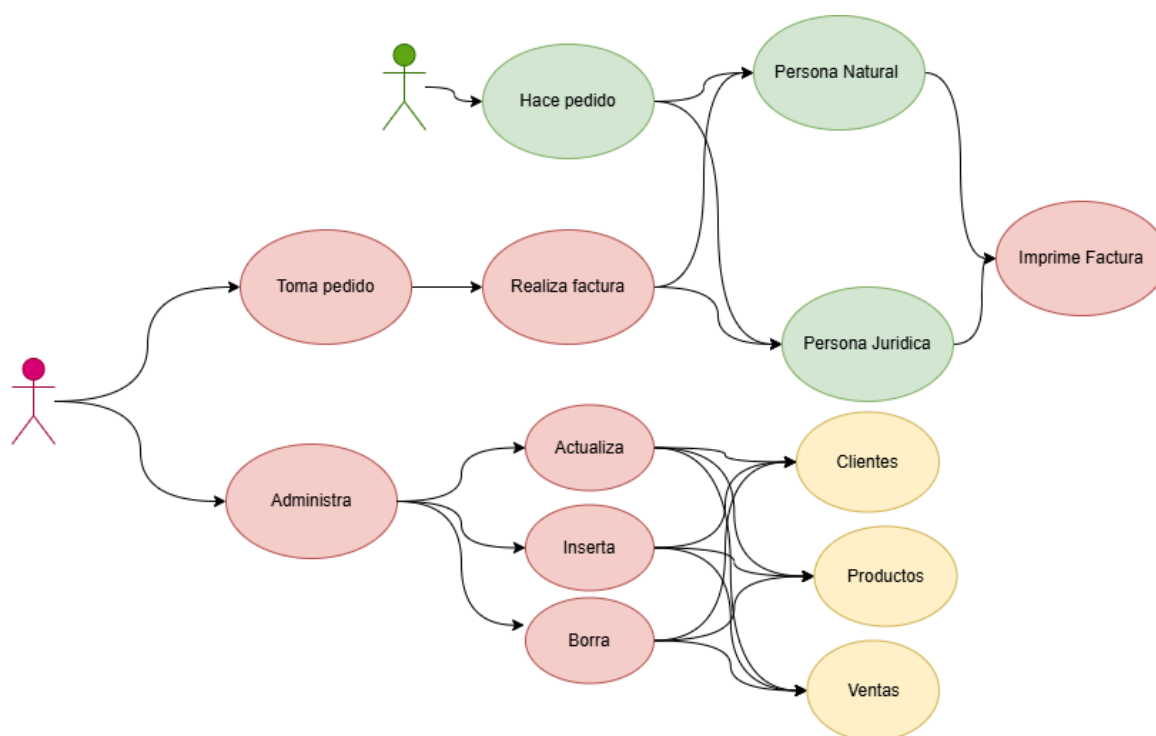


Imagen Clase de usos

El diagrama de clases fue elaborado utilizando la notación UML, ampliamente utilizada para representar sistemas orientados a objetos (Booch et al., 2005).

1.4 Descripción de las clases principales

Conexiones

La clase Conexiones establece la comunicación entre la aplicación y la base de datos MySQL mediante JDBC. Contiene el método conectar(), el cual crea y retorna un objeto Connection utilizando la base de datos "producción". Todas las operaciones de consulta, inserción, actualización y eliminación de registros utilizan esta clase para acceder a la información almacenada.

La conexión se implementó mediante la API JDBC, la cual proporciona una interfaz estándar para acceder a bases de datos desde aplicaciones Java (Oracle, 2025).

```
package conexiones;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexiones {
    private static final String url = "jdbc:mysql://localhost:3306/produccion";
    private static final String usuario = "root";
    private static final String pass = "admin";

    public static Connection conectar() {
        Connection conexion = null;
        try {
            conexion = DriverManager.getConnection(url, user:usuario, password: pass);
            System.out.println("Conexion correcta a Mysql");
        } catch (Exception e) {
            System.out.println("Error de Conexion " + e);
        }
        return conexion;
    }

    public static void main(String[] args) {
        Conexiones test = new Conexiones();
        test.conectar();
    }
}
```

Su función principal es establecer y gestionar el enlace físico de la aplicación Java con el servidor de base de datos MySQL, encapsulando los parámetros de autenticación y retornando un objeto reutilizable de tipo Connection.

Parámetros de Configuración (Constantes Privadas)

El uso de modificadores `private static final` garantiza que los credenciales no se puedan modificar externamente en tiempo de ejecución:

- **url (jdbc:mysql://localhost:3306/produccion):** Define la cadena de conexión JDBC (*Java Database Connectivity*).
 - `jdbc:mysql://`: Protocolo de conexión.
 - `localhost:3306`: Host local y puerto por defecto de MySQL.
 - `/produccion`: Nombre del esquema / base de datos del sistema.
- **usuario / pass:** Credenciales de acceso con permisos sobre la base de datos (root / admin).

public static Connection conectar(): Al ser un método estático (static), permite ser invocado desde cualquier otra clase del sistema (como VentanaClientes, VentanaProductos, etc.) sin necesidad de crear múltiples instancias de la clase Conexiones (Conexiones.conectar()).

Control de Excepciones (try-catch):

- Si la conexión es exitosa, imprime un mensaje de confirmación en la consola de desarrollo y retorna el objeto conexion.
- Si hay un fallo (ej. servidor apagado, clave incorrecta o base de datos no encontrada), atrapa la excepción Exception, imprime el detalle del error en consola y retorna null.

Clases interfaces gráficas:

- **VentanaLogin**
La clase VentanaLogin es la encargada de autenticar a los usuarios antes de permitir el acceso al sistema. Verifica las credenciales ingresadas y, cuando son correctas, abre la ventana principal de la aplicación.
- **VentanaPrincipal**
La clase VentanaPrincipal funciona como el menú principal del sistema. Desde esta ventana el usuario puede acceder a los diferentes módulos disponibles, como la administración de clientes, productos y pedidos, centralizando toda la navegación de la aplicación.
- **VentanaClientes**
La clase VentanaClientes permite administrar la información de los clientes registrados en la base de datos. Desde esta ventana se pueden consultar, registrar, modificar y eliminar clientes utilizando los formularios correspondientes.
- **VentanaProductos**
La clase VentanaProductos muestra el inventario de productos almacenados en la base de datos. La información se presenta en una tabla que facilita la consulta de los artículos disponibles en la ferretería.
- **VentanaAdminProductos**
La clase VentanaAdminProductos reúne las funciones administrativas relacionadas con los productos. Permite registrar nuevos artículos, modificar la información existente y eliminar productos cuando sea necesario.
- **VentanaPedido**
La clase VentanaPedido administra el proceso de creación de pedidos. En ella se seleccionan los productos, se calculan los valores correspondientes y se aplican los diferentes tipos de pedidos y descuentos implementados dentro del proyecto.

(Nota: En la sección de *Interfaz Gráfica* se analiza a detalle la implementación y el código fuente correspondiente a cada ventana del sistema.)

Clases de POO interfaz, herencia, polimorfismo...

1. Class Pedido

La clase Pedido actúa como la **clase base (padre)** para el modelado de transacciones en la ferretería. Al ser declarada como abstract, no se puede instanciar directamente; en su lugar, define la estructura común, el almacenamiento en memoria del carrito de compras y el contrato de métodos que deben implementar los distintos tipos de pedidos (por ejemplo, *Pedido Contado*, *Pedido Crédito* o *Pedido Mayorista*). Además, implementa la interfaz *Imprimible*.

```
public abstract class Pedido implements Imprimible {
    private int idCliente; // <-- Nuevo atributo encapsulado para la BD
    private String cliente;
    protected List<String> productos;
    protected List<Integer> idProductos;
    protected List<Double> precios;
    protected List<Integer> cantidades;
    protected double subtotal;
    protected String fecha;

    public Pedido(int idCliente, String cliente, String fecha) {
        this.idCliente = idCliente;
        this.cliente = cliente;
        this.productos = new ArrayList<>();
        this.idProductos = new ArrayList<>();
        this.precios = new ArrayList<>();
        this.cantidades = new ArrayList<>();
        this.subtotal = 0.0;
        this.fecha = fecha;
    }

    public void agregarProducto(int idProducto, String producto, double precio, int cantidad) {
        this.idProductos.add(e: idProducto);
        this.productos.add(e: producto);
        this.precios.add(e: precio);
        this.cantidades.add(e: cantidad);
        this.subtotal += (precio * cantidad);
    }
}
```

Encapsulamiento y Manejo de Estructuras Dinámicas

- **Atributos private y protected:**
 - Se encapsulan los datos sensibles del cliente (idCliente, cliente) para protegerlos del acceso no controlado.
 - Los atributos del detalle de la venta (productos, idProductos, precios, cantidades) utilizan el modificador protected para permitir que las clases hijas derivadas accedan a ellos directamente.
- **Uso de Listas Dinámicas (ArrayList):** En lugar de arreglos estáticos, emplea la interfaz List con ArrayList para permitir que un pedido contenga un número indeterminado de productos sin restricciones de tamaño fijo.

Abstracción y Contratos (abstract / Imprimible)

- **public abstract double calcularTotalFinal();** Fuerza a las clases hijas a definir su propia regla de cálculo del total (por ejemplo, aplicando impuestos, recargos por crédito o descuentos específicos).
- **public abstract String obtenerTipoPedido();** Método abstracto que obliga a cada subtipo a identificarse explícitamente.
- **implements Imprimible:** Garantiza que cualquier pedido derivado cumpla con el contrato de impresión requerido por el sistema.

```

public abstract double calcularTotalFinal();
public abstract String obtenerTipoPedido();

// Getters y Setters
public int getIdCliente() { return idCliente; }
public String getCliente() { return cliente; }
public double getSubtotal() { return subtotal; }
public List<String> getProductos() { return productos; }
public List<Integer> getIdProductos() { return idProductos; }
public List<Double> getPrecios() { return precios; }
public List<Integer> getCantidades() { return cantidades; }
public String getFecha() { return fecha; }

```

Acceso Seguro mediante Getters

- Proporciona métodos de lectura (`getIdCliente()`, `getSubtotal()`, `getProductos()`, etc.) para extraer la información de forma segura sin exponer las variables originales a modificaciones directas desde el exterior.

2. Class PedidoLocal

La clase `PedidoLocal` extiende de la clase abstracta `Pedido`. Representa las ventas que se realizan de manera presencial en la tienda (con entrega o retiro en el mismo establecimiento). Su propósito principal es **concretar las reglas de negocio específicas** para las ventas locales: no aplica costes ni recargos adicionales sobre el subtotal y define un formato de impresión ligero tipo *ticket*.

```

public class PedidoLocal extends Pedido {

    public PedidoLocal(int idCliente, String cliente, String fecha) {
        super(idCliente, cliente, fecha);
    }

    @Override
    public double calcularTotalFinal() {
        return getSubtotal();
    }

    @Override
    public String obtenerTipoPedido() {
        return "Retiro en Local";
    }
}

```

Sobrescritura Polimórfica (@Override)

Al ser una subclase concreta de una clase abstracta, `PedidoLocal` está obligada a implementar los métodos abstractos heredados:

- **calcularTotalFinal():**
 - Implementa la regla de negocio para compras presenciales: **el total es equivalente al subtotal sin recargos** por envío, flete o servicios financieros adicionales.
- **obtenerTipoPedido():**

- Retorna la etiqueta identificadora "Retiro en Local", la cual permite al sistema clasificar la transacción en los reportes o en la base de datos.
- **generarDocumento()**(Cumplimiento de la Interfaz Imprimible):
 - Define la representación en texto del comprobante de venta.
 - Utiliza **String.join(", ", getProductos())** para formatear dinámicamente la lista de nombres de los productos en una sola línea separada por comas.
 - Muestra el desglose final con el ID del cliente, nombre, fecha y el total calculado

Herencia y Reutilización de Código (extends Pedido)

- **extends Pedido:** Indica que PedidoLocal hereda todas las propiedades, atributos (cliente, lista de productos, precios) y métodos definidos en la clase base.
- **Invocación al Constructor Padre (super(...)):**

```
@Override
public void generarDocumento() {
    System.out.println(x: "=====");
    System.out.println(x: "          TICKET DE VENTA (LOCAL)          ");
    System.out.println(x: "=====");
    System.out.println("ID Cliente: " + getIdCliente());
    System.out.println("Cliente: " + getCliente());
    System.out.println("Fecha: " + getFecha());
    System.out.println("Artículos: " + String.join(delimiter:", ", elements: getProductos()));
    System.out.println(x: "-----");
    System.out.println("TOTAL NETO: $" + calcularTotalFinal());
    System.out.println(x: "=====");
}
```

3. PedidoDomicilio

La clase PedidoDomicilio hereda de la clase abstracta Pedido y representa las compras que requieren despacho físico. Su objetivo principal es incorporar la regla de negocio para entregas fuera de la tienda: añade un estado propio (cargoEnvio) que afecta directamente la suma final de la compra y adapta el formato de impresión para detallar el desglose del costo de envío.

```

public class PedidoDomicilio extends Pedido {
    private double cargoEnvio;

    public PedidoDomicilio(int idCliente, String cliente, String fecha, double cargoEnvio) {
        super(idCliente, cliente, fecha);
        this.cargoEnvio = cargoEnvio;
    }

    @Override
    public double calcularTotalFinal() {
        return getSubtotal() + cargoEnvio;
    }

    @Override
    public String obtenerTipoPedido() {
        return "Envío a Domicilio";
    }

    @Override
    public void generarDocumento() {
        System.out.println("=====");
        System.out.println("          FACTURA DE ENTREGA (DOMICILIO)          ");
        System.out.println("=====");
        System.out.println("ID Cliente: " + getIdCliente());
        System.out.println("Cliente: " + getCliente());
        System.out.println("Fecha: " + getFecha());
        System.out.println("Articulos: " + String.join(delimiter: ", ", elements: getProductos()));
        System.out.println("-----");
        System.out.println("Subtotal: $" + getSubtotal());
    }
}

```

Extensión de Estado y Constructor (super)

- **Atributo Propio (cargoEnvio):** Se declara una variable privada adicional `cargoEnvio` de tipo `double` para almacenar la tarifa de flete asignada a la orden.

Sobrescritura Polimórfica (@Override) e Integración de Reglas de Negocio

- **calcularTotalFinal():**
 - Redefine el cálculo del cobro sumando el valor acumulado de los artículos (`getSubtotal()`) más la tarifa adicional de flete (`cargoEnvio`).
- **obtenerTipoPedido():**
 - Devuelve la etiqueta identificadora "Envío a Domicilio", diferenciando esta modalidad en las consultas SQL, reportes o interfaces del sistema.
- **generarDocumento() (Implementación de Imprimible):**
 - Adapta la salida impresa agregando el desglose transparente de la compra: expone de manera independiente el **Subtotal**, el **Cargo de Envío** y el **Total Completo**, proporcionando claridad financiera al cliente final.

4. Descuento

La clase Descuento es la base del **motor de promociones y tarifas** de la aplicación. Al ser una clase abstracta, sirve como una plantilla o contrato que no se puede instanciar directamente, pero que obliga a todas sus clases derivadas (como `DescuentoMayorista` o `DescuentoRegular`) a implementar sus propias políticas de precio según el volumen o tipo de venta.

```
public abstract class Descuento {
    protected double totalOriginal;

    public Descuento(double totalOriginal) {
        this.totalOriginal = totalOriginal;
    }

    public abstract double calcularTotalConDescuento();
    public abstract double getPorcentaje();
    public abstract String getTipo();
}
```

Estado Protegido y Encapsulamiento (protected)

- **protected double totalOriginal:** Al declarar el monto inicial de la venta como protected, la variable queda protegida contra accesos indebidos desde clases externas, pero permanece directamente accesible para cualquier subclase que herede de Descuento.

Inyección de Valor por Constructor

- **public Descuento(double totalOriginal):** Exige que al momento de aplicar una política de descuento, la clase reciba obligatoriamente el importe base sobre el cual se realizarán los cálculos de ajuste financiero.

Definición de Métodos Abstractos (Polimorfismo)

La clase establece tres métodos sin cuerpo (sin implementación), dejando la responsabilidad del algoritmo matemático a las subclases:

- **public abstract double calcularTotalConDescuento();**
 - Firma el contrato para que cada tipo de descuento aplique su propia fórmula matemática para reducir el total a pagar.
- **public abstract double getPorcentaje();**
 - Obliga a cada regla a reportar qué tasa de descuento porcentual se aplicó (ej. 0.0 para clientes regulares, 0.10 para mayoristas).
- **public abstract String getTipo();**
 - Requiere un identificador textual que describa la promoción aplicada (ej. "Sin Descuento", "Descuento Mayorista (10%)"), útil para mostrar en la factura o ticket impreso.

5. DescuentoMayorista

La clase DescuentoMayorista hereda directamente de la clase abstracta Descuento. Su función es **materializar la regla de negocio para compras de gran volumen**: cuando

el monto de la venta supera el umbral establecido por el negocio (por ejemplo, \$50,000), el sistema aplica de forma automática un **10% de rebaja** sobre el subtotal original.

```
public class DescuentoMayorista extends Descuento {
    public DescuentoMayorista(double totalOriginal) {
        super(totalOriginal);
    }

    @Override
    public double calcularTotalConDescuento() {
        return totalOriginal * 0.90; // Aplica un 10% de descuento
    }

    @Override
    public double getPorcentaje() { return 10.0; }

    @Override
    public String getTipo() { return "Promoción Mayorista"; }
}
```

Herencia y Delegación (super)

- **extends Descuento:** Indica que DescuentoMayorista adopta la estructura y atributos de la clase padre base Descuento.

Sobrescritura de Métodos (@Override)

- **calcularTotalConDescuento():**
 - Implementa el algoritmo matemático específico para ventas al mayor. Multiplica el importe base por 0.90 ($\text{totalOriginal} * 0.90$), lo que equivale a deducir el 10% del precio base.
- **getPorcentaje():**
 - Devuelve el valor numérico explícito (10.0), facilitando que otros módulos (como el de generación de reportes o la interfaz gráfica) puedan mostrar la tasa aplicada.
- **getTipo():**
 - Devuelve la descripción textual "Promoción Mayorista", requerida para detallar en la vista previa del cobro y en la impresión de la factura qué beneficio se le otorgó al cliente.

6. Class DescuentoRegular

La clase DescuentoRegular extiende de la clase abstracta Descuento. Representa la **política por defecto o tarifa estándar** del sistema. Se aplica cuando el monto de la compra no alcanza el umbral mínimo para aplicar una tarifa de mayoreo (por ejemplo, compras menores a \$50,000), manteniendo el precio original sin modificaciones ni rebajas.

```
public class DescuentoRegular extends Descuento {
    public DescuentoRegular(double totalOriginal) {
        super(totalOriginal);
    }

    @Override
    public double calcularTotalConDescuento() {
        return totalOriginal; // Sin descuento
    }

    @Override
    public double getPorcentaje() { return 0.0; }

    @Override
    public String getTipo() { return "Cliente Regular"; }
}
```

Herencia y Reutilización del Constructor (super)

- **extends Descuento:** Hereda los atributos y comportamientos de la clase padre base Descuento.

Sobrescritura de Métodos (@Override)

- **calcularTotalConDescuento():**
 - Implementa la regla para clientes regulares: retorna el valor de totalOriginal intacto, sin aplicar deducciones ni porcentajes.
- **getPorcentaje():**
 - Devuelve 0.0, señalando explícitamente a la interfaz gráfica y al sistema de reportes que no se aplicó ningún porcentaje de descuento.
- **getTipo():**
 - Retorna la etiqueta identificadora "Cliente Regular", utilizada para detallar el tipo de tarifa aplicada en el desglose final de la factura o ticket.

7. Class DocumentoFactura

La clase DocumentoFactura actúa como la **plantilla base para la generación de comprobantes impresos**. Al ser declarada como abstract, no se puede instanciar directamente. Se encarga de encapsular los metadatos comunes de la transacción (ID de factura, cliente, total y fecha) (*Ticket POS* o *Factura de Oficina*) a sus subclases concretas.

```

public abstract class DocumentoFactura implements Imprimible {
    // Encapsulamiento
    private String idFactura;
    private String cliente;
    private String total;
    private String fecha;

    public DocumentoFactura(String idFactura, String cliente, String total, String fecha) {
        this.idFactura = idFactura;
        this.cliente = cliente;
        this.total = total;
        this.fecha = fecha;
    }

    public String getIdFactura() { return idFactura; }
    public String getCliente() { return cliente; }
    public String getTotal() { return total; }
    public String getFecha() { return fecha; }

    public abstract String estructurarContenido();

    @Override
    public void generarDocumento() {
        String cuerpo = estructurarContenido();
        //simulamos el envio a la impresora
        javax.swing.JOptionPane.showMessageDialog(parentComponent: null, message: cuerpo, "Imprimiendo Factura N° " + idFactura,

```

Encapsulamiento y Protección de Datos

- **Atributos private:**
 - idFactura, cliente, total, fecha.
 - Mantener los atributos en ámbito privado garantiza la integridad de los datos de facturación, impidiendo que sean alterados una vez que el objeto ha sido construido.
- **Constructor e Inmutabilidad de Lectura:**
 - Se inicializa mediante el constructor DocumentoFactura(...) y solo se proveen métodos de acceso de lectura (Getters), asegurando que los datos del documento no se modifiquen durante el proceso de impresión.

Implementación de la Interfaz (implements Imprimible)

- Al implementar la interfaz Imprimible, la clase promete cumplir con el contrato de poseer un método generarDocumento(), permitiendo que el sistema procese cualquier tipo de factura de forma polimórfica

Método Abstracto y Patrón Template Method

- **public abstract String estructurarContenido();:**
 - Es el método clave de abstracción. Obliga a cada subtipo (por ejemplo, FacturaTicket o FacturaOficina) a definir cómo se debe maquetar el texto y las secciones del documento.
- **public void generarDocumento():**
 - Centraliza la ejecución de la "impresión". Invoca a estructurarContenido() para obtener la cadena con el formato armado según la subclase activa y simula la salida física a impresora desplegando un diálogo nativo (JOptionPane.showMessageDialog).

8. FacturaTicket

La clase FacturaTicket hereda de la clase abstracta DocumentoFactura. Su propósito es **definir la maquetación visual reducida (tipo *Ticket POS*)** para las impresoras térmicas de punto de venta. Esta plantilla compacta presenta únicamente los datos indispensables de la transacción (encabezado comercial, ID de comprobante, fecha, cliente, total pagado y mensaje de agradecimiento).

```
public class FacturaTicket extends DocumentoFactura {

    public FacturaTicket(String idFactura, String cliente, String total, String fecha) {
        super(idFactura, cliente, total, fecha); // Herencia
    }

    @Override
    public String estructurarContenido() {
        return "=====\n" +
            "    FERRETERÍA LA SOLUCIÓN  \n" +
            "=====\n" +
            "TICKET DE VENTA N°: " + getIdFactura() + "\n" +
            "Fecha: " + getFecha() + "\n" +
            "Cliente: " + getCliente() + "\n" +
            "-----\n" +
            "TOTAL PAGADO: $" + getTotal() + "\n" +
            "=====\n" +
            "    ¡Gracias por su compra!  \n" +
            "=====";
    }
}
```

Herencia y Reutilización del Constructor (super)

- **extends DocumentoFactura:** Adopta el comportamiento base y el estado de la clase padre de facturación.

Sobrescritura Polimórfica (@Override)

- **public String estructurarContenido():**
 - Cumple con el contrato definido por la clase padre implementando el método abstracto estructurarContenido().
 - Diseña y retorna una cadena formateada con caracteres ASCII (==, ---) y saltos de línea (\n), simulando la estética angosta y compacta típica de un ticket impreso en papel térmico.
 - Recupera los valores de la transacción de forma segura utilizando los métodos de acceso heredados (getIdFactura(), getFecha(), getCliente(), getTotal()).

9. FacturaOficina

La clase FacturaOficina hereda de la clase abstracta DocumentoFactura. Su propósito es **definir la maquetación visual extendida para comprobantes fiscales y administrativos**.

```
public class FacturaOficina extends DocumentoFactura {

    public FacturaOficina(String idFactura, String cliente, String total, String fecha) {
        super(idFactura, cliente, total, fecha);
    }

    @Override
    public String estructurarContenido() {

        return "-----\n" +
            "                FACTURA DE VENTA LEGAL                \n" +
            "-----\n" +
            "Código Registro: REG-" + getIdFactura() + "026\t\tEmisión: " + getFecha() + "\n" +
            "Adquiriente: " + getCliente().toUpperCase() + "\n" +
            "Régimen Comercial - Impuestos Incluidos\n" +
            "-----\n" +
            "VALOR NETO OPERACIÓN:\t\t\t$" + getTotal() + "\n" +
            "-----\n" +
            "Representación gráfica digitalizada de la transacción ferretera.\n";
    }
}
```

Herencia y Reutilización del Constructor (super)

- **extends DocumentoFactura:** Reutiliza la infraestructura base de la clase padre de facturación.

Sobrescritura Polimórfica (@Override) y Formateo Formal

- **public String estructurarContenido():**
 - Implementa la versión extendida del método abstracto `estructurarContenido()`.
 - **Tratamiento de Datos:** Aplica transformaciones a la información, como generar un código fiscal personalizado ("REG-" + `getIdFactura()` + "026")

10. Imprimible

La interfaz Imprimible define un **contrato genérico de salida de información** dentro de la arquitectura del sistema. Al ser una interface, no contiene implementación ni variables de estado; en su lugar, establece el método abstracto `generarDocumento()`, el cual debe ser obligatoriamente implementado por cualquier clase que requiera transformar o emitir sus datos hacia una representación visual o de impresión (como las distintas clases de *Pedidos* o de *Facturas*).

```
package clases;

public interface Imprimible {
    void generarDocumento();
}
```

11. Actualizador

La clase abstracta Actualizador sirve como base para los procesos de actualización de información. Declara el método solicitarNuevosDatos(), el cual es implementado por las clases derivadas según el tipo de actualización que se deba realizar.

```
public abstract class Actualizador {
    protected DatosFactura datos;

    public Actualizador(DatosFactura datos) {
        this.datos = datos;
    }

    public abstract boolean solicitarNuevosDatos();
}
```

12. Class ActualizadorFormulario

La clase ActualizadorFormulario hereda de la clase abstracta Actualizador. Su propósito es **gestionar la edición dinámica de los metadatos de una factura** (cliente y fecha) mediante cuadros de diálogo interactivos nativos (JOptionPane).

```
public class ActualizadorFormulario extends Actualizador {

    public ActualizadorFormulario(DatosFactura datos) {
        super(datos);
    }

    @Override
    public boolean solicitarNuevosDatos() {
        // Modificar Nombre del Cliente
        String nuevoCliente = JOptionPane.showInputDialog(parentComponent: null,
            message: "Modificar nombre del cliente o Razón Social:", initialSelectionValue: datos.getCliente());

        if (nuevoCliente == null) return false; //
        if (!nuevoCliente.trim().isEmpty()) {
            datos.setCliente(cliente: nuevoCliente.trim());
        }

        // Modificar Fecha de Emisión
        String nuevaFecha = JOptionPane.showInputDialog(parentComponent: null,
            message: "Modificar fecha de emisión (AAAA-MM-DD):", initialSelectionValue: datos.getFecha());

        if (nuevaFecha == null) return false;
        if (!nuevaFecha.trim().isEmpty()) {
            datos.setFecha(fecha: nuevaFecha.trim());
        }

        return true;
    }
}
```

Herencia y Reutilización de Estado (super)

- **extends Actualizador:** Adopta el contrato y el objeto contenedor de datos de la clase base.

Sobrescritura (@Override) y Flujo de Interacción

Implementa el método abstracto `solicitarNuevosDatos()` para guiar al usuario a través de un flujo secuencial en pantalla:

1. Captura del Cliente / Razón Social/ Fecha

- Muestra un cuadro de diálogo con `JOptionPane.showInputDialog(...)` inicializado con el valor actual (`datos.getClientes()`).
- **Control de Cancelación:** Si el usuario presiona "Cancelar" o cierra la ventana (`nuevoCliente == null`), interrumpe la secuencia inmediatamente devolviendo `false`.
- **Sanitización e Inyección:** Utiliza `.trim().isEmpty()` para evitar guardar textos en blanco. Si la entrada es válida, actualiza la propiedad mediante `datos.setCliente(...)`.
- Solicita la nueva fecha mostrando el formato requerido (AAAA-MM-DD).
- Repite la verificación de cancelación (`nuevaFecha == null`) y sanitización antes de aplicar `datos.setFecha(...)`.

CAPÍTULO II. INTERFAZ GRÁFICA

2.1 Interfaz de usuario

Se ha creado una interfaz gráfica con Java para la interacción del sistema con los usuarios a través de ventanas, botones, menús, tablas y formularios, los usuarios pueden acceder a las diferentes funcionalidades del programa, como la gestión de clientes, empleados, inventario y pedidos. Su diseño busca facilitar la navegación, mejorar la experiencia de uso y optimizar la ejecución de las tareas administrativas dentro de la ferretería. La interfaz gráfica fue desarrollada utilizando la biblioteca Java Swing, la cual permite construir aplicaciones de escritorio mediante componentes gráficos reutilizables (Oracle, 2025)

Pantalla de inicio

Este módulo gestiona la **autenticación y acceso al sistema** de la ferretería. Se encarga de validar las credenciales de un usuario contra la base de datos, aplicar medidas de seguridad como la prevención de **Inyección SQL**, controlar un **límite de intentos fallidos** para evitar accesos no autorizados y realizar el redireccionamiento a la ventana principal del sistema (VentanaPrincipal).



Flujo de Autenticación:

El bloque `try-catch` permite gestionar los errores del sistema de forma controlada. Por un lado, valida el cumplimiento de las condiciones requeridas para el usuario; por otro, captura fallos internos como pérdidas de conexión a la base de datos. Ante una excepción no controlada, el sistema emite un mensaje de error detallado útil para el desarrollador, garantizando la estabilidad de la aplicación.

1. El usuario ingresa sus credenciales y presiona el botón **Acceder**.
2. El sistema realiza una consulta a la base de datos para autenticar la cuenta.
3. **Resultado Exitoso:** Se despliega una alerta de confirmación ("Datos correctos") y se concede el acceso.
4. **Resultado Fallido:** Se muestra un mensaje de error ("Usuario o contraseña incorrectos") notificando el fallo de autenticación.
5. En caso de ingresar datos erróneos, se notificará al usuario el fallo. Si se acumulan **tres intentos fallidos consecutivos**, el sistema bloqueará la cuenta automáticamente por seguridad, obligando al usuario a contactar con soporte técnico para recuperar el acceso.

6. Al seleccionar el botón **Salir**, la aplicación finaliza de manera segura la sesión activa, libera los recursos del sistema y concluye la ejecución del programa mediante el cierre de la interfaz gráfica

```
private void b_accederActionPerformed(java.awt.event.ActionEvent evt) {
    Connection conexion = Conexiones.conectar();
    String usuario = campo_usuario.getText();
    String clave = new String(value: campo_pass.getPassword());
    PreparedStatement consulta;
    ResultSet resultado;
    int control = 0;
    try{
        String sql = "select * from usuarios where nombre_usuario = ? and clave = ?";
        consulta = conexion.prepareStatement(string:sql);
        consulta.setString(1, string:usuario);
        consulta.setString(2, string:clave);
        resultado = consulta.executeQuery();

        if(campo_usuario.getText().isEmpty() || new String(value: campo_pass.getPassword()).isEmpty()){
            JOptionPane.showMessageDialog(parentComponent: null,message: "No debe haber campos vacios", title: "ERROR",
            messageType: JOptionPane.ERROR_MESSAGE);
            return;
        }

        if(resultado.next()){
            JOptionPane.showMessageDialog(parentComponent: null,message: "Login Correcto", title: "LOGIN",
            messageType: JOptionPane.INFORMATION_MESSAGE);
            VentanaPrincipal principal = new VentanaPrincipal(); // Instancias tu ventana principal
            principal.setVisible(= true); // La haces visible
            this.dispose();
        }else{
            intentos++;
            JOptionPane.showMessageDialog(parentComponent: null, "Usuario/Contraseña incorrectos " + "Intente de nuevo", title: "ERROR",
            messageType: JOptionPane.ERROR_MESSAGE);
            campo_usuario.setText("");
        }
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(parentComponent: null, "Error de conexión: " + ex.getMessage(), title: "ERROR",
        messageType: JOptionPane.ERROR_MESSAGE);
    }
}
```

Código de acceso al sistema

Captura de Datos y Seguridad Previa

- **campo_pass.getPassword():** Captura la contraseña retornando un arreglo de caracteres (char []), el cual se convierte a String.
- **Consultas Preparadas (PreparedStatement):**

Validación de Campos Vacíos

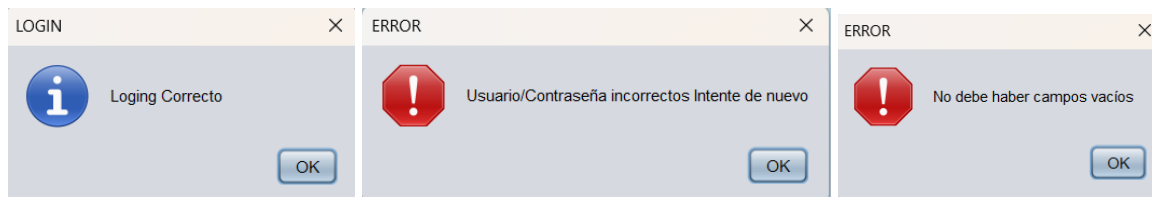
- **campo_usuario.getText().isEmpty() || ...:** Verifica que las cajas de texto no se envíen vacías. Si alguna lo está, interrumpe el flujo (return) e informa al usuario mediante una alerta (ERROR_MESSAGE).

Control de Acceso y Redirección

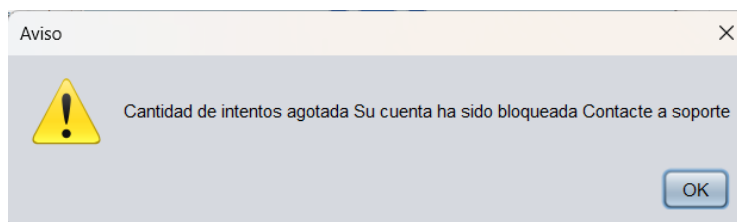
- **resultado.next():** Evalúa si la base de datos devolvió un registro coincidente:
 - **Acceso Exitoso:**
 1. Despliega un mensaje de bienvenida.
 2. Instancia la ventana de navegación principal (new VentanaPrincipal().setVisible(true)).
 3. Cierra la ventana actual de Login (this.dispose()) para liberar memoria.
 - **Acceso Fallido:**
 1. Incrementa la variable acumuladora intentos++.
 2. Notifica el error de credenciales mediante un diálogo visual.
 3. Limpia los campos de texto (setText(" ")) para reiniciar el ingreso.

Política de Intentos Agotados (Seguridad)

- **if(intentos == 3):** Evalúa si el usuario ha fallado tres veces consecutivas. De ser así, activa una alerta de seguridad advirtiéndole que la cuenta/acceso ha sido bloqueado y solicita contactar al equipo de soporte.



Mensajes de error e información al usuario.



Mensaje de usuario bloqueado

2.2 Módulos principales

Después de iniciar sesión correctamente, el usuario accede a la ventana principal del sistema, desde la cual puede ingresar a los diferentes módulos Clientes, Productos y Ventas.



Imagen ventana principal

La ventana principal del sistema consta de un menú con **cinco botones de navegación**:

- **Módulos de Gestión (3):** Permiten acceder de forma independiente a la administración de **Clientes**, **Productos** y **Ventas** de la ferretería:
- **Navegación y Control (2):** El botón **Volver** permite regresar a la pantalla anterior, mientras que el botón **Salir** cierra la sesión en el módulo de administración y redirige al usuario a la interfaz de *Login*.

Distribución de Botones - Panel Principal:

1. **Módulo Clientes:** Acceso al registro y consulta de clientes.
2. **Módulo Productos:** Gestión del catálogo de inventario.
3. **Módulo Ventas:** Procesamiento de transacciones de la ferretería.

4. **Volver:** Retorno a la pantalla jerárquicamente superior.
5. **Salir:** Cierre de sesión del área administrativa y retorno al *Login*.

```

private void b_clientesActionPerformed(java.awt.event.ActionEvent evt) {
    dispose();
    new VentanaClientes().setVisible(b: true);
}

private void b_salirActionPerformed(java.awt.event.ActionEvent evt) {
    VentanaLogin menu = new VentanaLogin();
    menu.setVisible(b: true);
    this.dispose();
}

private void b_volverActionPerformed(java.awt.event.ActionEvent evt) {
    VentanaPrincipal menu = new VentanaPrincipal();
    menu.setVisible(b: true);
    this.dispose();
}

private void b_ventasActionPerformed(java.awt.event.ActionEvent evt) {
    dispose();
    new VentanaVentas().setVisible(b: true);
}

private void b_productosActionPerformed(java.awt.event.ActionEvent evt) {
    dispose();
    new VentanaProductos().setVisible(b: true);
}

```

Código botones ventana principal.

Este segmento de código gestiona la redirección del usuario a través de la interfaz. Al hacer clic en cualquiera de los botones principales, el sistema ejecuta un evento de navegación que abre la ventana del módulo correspondiente (**Cientes**, **Productos** o **Ventas**) para la administración del negocio.

Redirección a Módulos Operativos

- **b_clientesActionPerformed, b_ventasActionPerformed y b_productosActionPerformed:**
 - **new VentanaX().setVisible(true):** Instancian y despliegan la pantalla correspondiente al módulo seleccionado por el usuario.
 - **dispose():** Libera los recursos de la ventana actual y la cierra de la memoria gráfica, asegurando que solo permanezca activa la nueva interfaz abierta.

Control de Sesión y Retorno

- **b_salirActionPerformed (Cerrar Sesión):**
 - Finaliza la sesión actual del usuario, instancia la ventana de autenticación (VentanaLogin) y destruye la interfaz actual mediante `this.dispose()`.
- **b_volverActionPerformed (Navegación Atrás):**
 - Permite regresar desde cualquier subpantalla o módulo secundario hacia el menú principal (VentanaPrincipal), manteniendo un patrón consistente de navegación en la interfaz.

Módulo de Clientes

Permite registrar, consultar, actualizar y eliminar la información de los clientes almacenados en la base de datos. En la sección de base de datos de este informe se puede ver cómo está relacionada la información de los clientes con otros módulos del sistema.



Para la administración de la información de los clientes, cada botón de la interfaz dirige al usuario a una ventana específica que permite realizar las operaciones correspondientes sobre la base de datos. A través de estas ventanas es posible registrar nuevos clientes, actualizar la información existente y eliminar registros cuando sea necesario, garantizando una gestión eficiente y organizada de los datos almacenados.

```
public void BuscarNombre () {
    String Nombrecliente = t_nombre.getText();
    if(Nombrecliente.isEmpty()) {
        JOptionPane.showMessageDialog(parentComponent: null, message: "Debe ingresar nombre de usuario",
            return;
    }
    String sql = "select * from clientes where nombre = ?";
    DefaultTableModel modelo = (DefaultTableModel)tabla1.getModel();
    modelo.setRowCount(0);

    public void Buscar_Id_Nombre () {
        String idCliente = t_cliente.getText();

        if(idCliente.isEmpty()) {
            JOptionPane.showMessageDialog(parentComponent: null, "Ingrese Id del cliente "
                return;
        }
        String sql = "select * from clientes where id_cliente=?";
```

Este bloque abarca la **inicialización gráfica del módulo de clientes** y dos métodos para la **filtración de información**: la búsqueda exacta por identificador único (Buscar_Id_Nombre) y la búsqueda por coincidencia de nombre (BuscarNombre).

Ambos procedimientos consultan la base de datos MySQL de forma segura y **refrescan dinámicamente la tabla de la interfaz** (tabla1) con los resultados obtenidos.

Consulta SQL Parametrizada: `SELECT * FROM clientes WHERE id_cliente = ?`

Si la consulta devuelve datos (`resultado.next()`), extrae los campos (`id_cliente`, `nombre`, `direccion`, `telefono`) por el nombre de la columna y los inserta como una nueva fila. Si el ID no existe en la base de datos, notifica al usuario con un mensaje flotante.

```

26 public void Mostrar(String tabla) {
27     String sql = "select * from " + tabla;
28     Connection con = conexion.conectar();
29
30     DefaultTableModel modelo = new DefaultTableModel(
31         new Object[][](),
32         new String[]{"Id_cliente", "Nombre", "Direccion", "Telefono"}
33     ) {
34         @Override
35         public boolean isCellEditable(int row, int column) {
36             return false; // Al retornar false, bloqueas la edición de todas las celdas
37         }
38     };
39
40     tabla1.setModel(dataModel:modelo);
41
42     for (int f = 0; f < tabla1.getColumnCount(); f++) {
43         tabla1.getColumnModel().getColumn(columnIndex: f).setResizable(false);
44     }
45     String[] datos = new String[4];
46
47     try {
48         Statement consulta = con.createStatement();
49         ResultSet resultado = consulta.executeQuery(string:sql);
50
51         while(resultado.next()){
52             datos[0] = resultado.getString(1);
53             datos[1] = resultado.getString(2);
54             datos[2] = resultado.getString(3);
55             datos[3] = resultado.getString(4);
56             modelo.addRow(rowData: datos);

```

Método para mostrar tabla clientes

El método `Mostrar` se encarga de consultar la base de datos MySQL mediante la conexión activa para **extraer e ingresar los registros de clientes directamente en un componente visual JTable** (tabla1). Además, aplica reglas de diseño para restringir la edición directa y evitar que el usuario altere el ancho de las columnas.

Funcionalidades del Módulo de Clientes:

- **Botón Buscar:** un botón para buscar por nombre o id del cliente dentro de la tabla clientes
- **Botón Guardar / Insertar:** Registra un nuevo cliente en la base de datos.
- **Botón Actualizar / Modificar:** Aplica los cambios realizados sobre la información de un cliente existente.
- **Botón Eliminar / Suprimir:** Remueve permanentemente el registro seleccionado de la base de datos.

El sistema fue diseñado con mecanismos de validación de datos que impiden dejar campos obligatorios en blanco y verifican que la información ingresada cumpla con el formato y los caracteres permitidos. Estas restricciones contribuyen a mantener la integridad, consistencia y confiabilidad de la base de datos, evitando errores, registros incompletos o modificaciones incorrectas en la información almacenada.

La eliminación **física de un cliente** de la base de datos a partir de su identificador único (id_cliente). El procedimiento incluye una **validación numérica previa** de la entrada de texto, la ejecución segura de la sentencia DELETE mediante consultas preparadas,

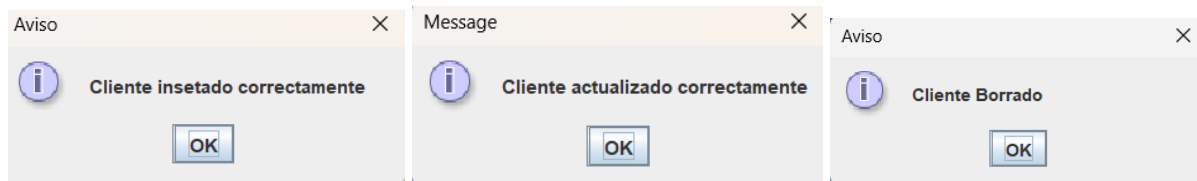
```

119 private void b_insertarActionPerformed(java.awt.event.ActionEvent evt) {
120     String nombre = t_nombre.getText().trim();
121     String direccion = t_direccion.getText().trim();
122     String telefono = t_telefono.getText().trim();
123
124     Conexiones conexion = new Conexiones();
125     Connection con = conexion.conectar();
126
127     String sql = "insert into clientes (nombre, direccion, telefono) values (?, ?, ?)";
128     if(nombre.isEmpty() || direccion.isEmpty() || telefono.isEmpty()){
129         JOptionPane.showMessageDialog(parentComponent: null, message: "Debes llenar todos los campos", title: "Alerta",
130                                     messageType: JOptionPane.WARNING_MESSAGE);
131         return;
132     }
133     try{
134         PreparedStatement insertar = con.prepareStatement(string:sql);
135         insertar.setString(1, string:nombre);
136         insertar.setString(2, string:direccion);
137         insertar.setString(3, string:telefono);
138
139         int resultado = insertar.executeUpdate();
140         if(resultado > 0){
141             JOptionPane.showMessageDialog(parentComponent: null, message: "Cliente insertado correctamente ", title: "Aviso",
142                                         messageType: JOptionPane.INFORMATION_MESSAGE);
143         }
144     }catch(SQLException e){
145         JOptionPane.showMessageDialog(parentComponent: null, message: "Error en el Insert ", title: "Alerta",
146                                     messageType: JOptionPane.WARNING_MESSAGE);
147     }
148 }

```

Código base para insertar, actualizar suprimir un cliente

Cada Ventana de la administración del módulo clientes Incluye la captura y limpieza de los datos ingresados por el usuario, una validación preventiva de campos vacíos, la ejecución segura de la consulta mediante consultas preparadas (PreparedStatement) y el despliegue de notificaciones dinámicas (JOptionPane).



Módulo de Productos

Facilita la administración del catálogo de productos, permitiendo registrar nuevos artículos, modificar la información existente, realizar consultas y eliminar registros cuando sea necesario.

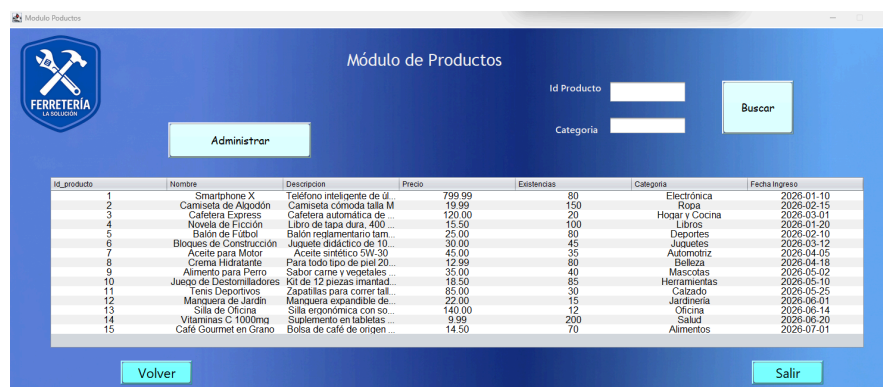
```

public void Mostrar(String tabla) {
    String sql = "select p.id_producto, p.nombre, p.descripcion, p.precio, p.existencia, c.nombre_categoria, p.fecha_ingreso " +
                "from " + tabla + " p " +
                "join categorias c on p.id_categoria = c.id_categoria";

    DefaultTableModel modelo = new DefaultTableModel(
        new Object[][] {},
        new String[] {"Id_producto", "Nombre", "Descripcion", "Precio", "Existencias", "Categoria", "Fecha Ingreso"}
    );
}

```

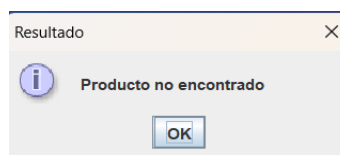
Debido a la estructura relacional de la base de datos, la tabla productos solo almacena el `id_categoria`. Para obtener y mostrar el nombre completo de la categoría asociada a cada producto, el sistema ejecuta una consulta mediante la cláusula `JOIN` entre las tablas `productos` y `categorias`, y se muestran cada columna en la ventana de Productos ya conociendo a qué familia pertenece ese producto.



El botón "Administrar" del módulo de productos permite gestionar la información almacenada en la base de datos mediante las operaciones CRUD (Crear, Leer, Actualizar y Eliminar). A través de esta funcionalidad, el usuario puede registrar nuevos productos, consultar la información existente, actualizar o corregir datos de productos ya registrados y eliminar aquellos que ya no se encuentren disponibles, garantizando así la integridad y actualización constante del inventario.

Cada botón de la interfaz gráfica está asociado a un controlador de eventos específico. Al interactuar con ellos, el sistema ejecuta sentencias SQL de manipulación de datos (DML) para realizar las operaciones de **inserción** (INSERT), **actualización** (UPDATE) o **eliminación** (DELETE) directamente sobre la base de datos.

1. Se busca por el id del producto para que cuando se presione el botón buscar traiga los datos asociados a ese producto, sino existe el id producto el sistema manda un mensaje de “Producto no encontrado”



```
try{
    Connection con = conexion.conectar();
    PreparedStatement consultar = con.prepareStatement(string:sql);

    consultar.setInt(1, 1, idProductoInt);
    ResultSet resultado = consultar.executeQuery();

    if(resultado.next()){
        t_idProducto.setText(t: String.valueOf(i: resultado.getInt(string:"id_producto")));
        t_nombre.setText(t: String.valueOf(obj: resultado.getString(string:"nombre")));
        t_descripcion.setText(t: String.valueOf(obj: resultado.getString(string:"descripcion")));
        t_precio.setText(t: String.valueOf(d: resultado.getDouble(string:"precio")));
        t_existencia.setText(t: String.valueOf(i: resultado.getInt(string:"existencia")));
        t_categoria.setText(t: String.valueOf(obj: resultado.getString(string:"nombre_categoria")));
        t_fecha_ingreso.setText(t: String.valueOf(obj: resultado.getDate(string:"fecha_ingreso")));
    }else{
        JOptionPane.showMessageDialog(parentComponent: null,message: "Producto no encontrado", title: "Resultado",
            messageType: JOptionPane.INFORMATION_MESSAGE);
    }
}
```

Código que trae información del producto que se busca

2. Una vez cargada la información del producto mediante la búsqueda previa, el usuario puede ejecutar la **actualización** o la **eliminación** del registro en la base de datos al presionar el botón correspondiente a la acción deseada.

```
String sql = "UPDATE productos SET nombre = ?, precio = ?, existencia = ? WHERE id_producto = ?";

try {
    Conexiones conexion = new Conexiones();
    Connection con = conexion.conectar();
    PreparedStatement consulta = con.prepareStatement(string:sql);

    // parámetros a la consulta
    consulta.setString(i: 1, string:nombre);
    consulta.setDouble(i: 2, d: Double.parseDouble(s: precio));
    consulta.setInt(i: 3, i1: Integer.parseInt(s: existencia));
    consulta.setInt(i: 4, i1: Integer.parseInt(s: id));

    // actualización en la BD
    int filasAfectadas = consulta.executeUpdate();
}
```

Código actualizar datos

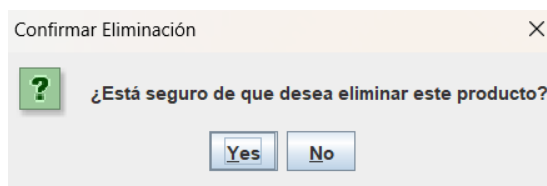
```
int confirmar = JOptionPane.showConfirmDialog(parentComponent: null, message: "¿Está seguro de que desea eliminar este producto?",
    title: "Confirmar Eliminación", optionType: JOptionPane.YES_NO_OPTION, messageType: JOptionPane.QUESTION_MESSAGE);

if (confirmar == JOptionPane.YES_OPTION) {
    try {
        int idProductoInt = Integer.parseInt(s: idProducto);
        Conexiones conexion = new Conexiones();
        Connection con = conexion.conectar();

        String sql = "DELETE FROM productos WHERE id_producto = ?";
        PreparedStatement consulta = con.prepareStatement(string: sql);
        consulta.setInt(i: 1, i1: idProductoInt);

        int filasEliminadas = consulta.executeUpdate();
        if (filasEliminadas > 0) {
            JOptionPane.showMessageDialog(parentComponent: null, message: "PRODUCTO ELIMINADO CORRECTAMENTE", title: "RESULTADO", messageType: JOptionPane.INFORMATION_MESSAGE);
        }
    }
}
```

Código de borrar datos producto



Confirmación de borrar producto

Procedimiento para Registro de Productos:

1. **Diligenciamiento de datos:** Completar las cajas de texto con la información del producto (nombre, precio, stock, categoría).
2. **Omisión de ID:** Dejar libre el campo *ID Producto*, ya que la base de datos lo genera en secuencia autoincrementable.
3. **Confirmación:** Presionar el botón **Insertar** para ejecutar la consulta SQL en la base de datos.

```
String sql = "INSERT INTO productos (nombre, descripcion, precio, existencia, id_categoria, fecha_ingreso) VALUES (?, ?, ?, ?, ?, ?)";

try {
    PreparedStatement consulta = con.prepareStatement(string:sql);

    consulta.setString(i: 1, string:nombre);
    consulta.setString(i: 2, string:descripcion);
    consulta.setDouble(i: 3, d: precio);
    consulta.setInt(i: 4, i1: existencia);
    consulta.setInt(i: 5, i1: idCategoria); // El número ID real encontrado
    consulta.setDate(i: 6, date: java.sql.Date.valueOf(date: fechaIngreso));

    int filasInsertadas = consulta.executeUpdate();
    if (filasInsertadas > 0) {
        JOptionPane.showMessageDialog(parentComponent: null, message: "PRODUCTO INSERTADO CORRECTAMENTE", title: "RESULTADO", messageType: JOptionPane.INFORMATION_MESSAGE);
    }
}
```

Código inserción de un nuevo producto

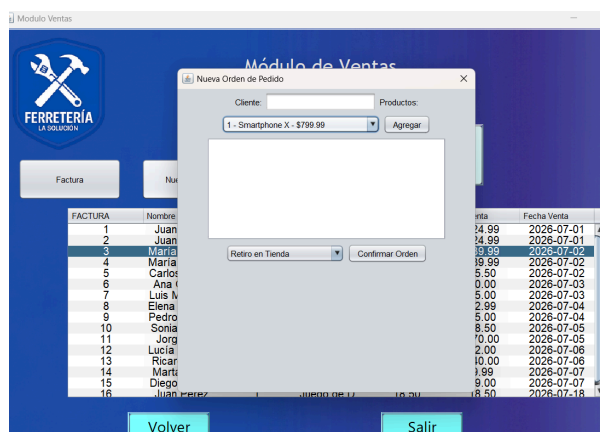
Modulo de Ventas

Permite registrar las ventas realizadas, consultar la información relacionada y almacenar los datos en la base de datos para mantener un control de las operaciones comerciales.

Además contiene un botón que permite imprimir factura que se quiere consultar y otro botón que genera un nuevo pedido dependiendo de las existencias en almacén.



El botón "Nuevo Pedido" permite registrar una nueva orden de compra y generar la factura correspondiente. A través de esta opción, el usuario puede agregar los artículos que el cliente desea adquirir, calculando automáticamente el valor total de la compra antes de emitir la factura. Además, esta ventana permite consultar la disponibilidad de los productos en el almacén, evitando la facturación de artículos sin existencias y garantizando la integridad de la información registrada en el sistema.



Los botones **Nuevo Pedido** y **Factura** fueron implementados aplicando principios de Programación Orientada a Objetos. Mediante el **encapsulamiento**, los datos de los pedidos y facturas se gestionan a través de métodos seguros (como *getters* y *setters*), evitando accesos no autorizados y mejorando la seguridad del código.

El bloque de código ejecuta el proceso de **facturación / nuevo pedido** mediante un esquema **transaccional en MySQL**. Primero inserta el registro principal de la venta (cabecera), recupera su **ID autogenerado** y, posteriormente, inserta múltiple información correspondiente a cada producto del pedido (detalle) utilizando **procesamiento por lotes**

```

conexiones.Conexiones conexion = new conexiones.Conexiones();
Connection con = conexion.conectar();

String sqlVenta = "INSERT INTO ventas (id_cliente, total_venta, fecha_venta) VALUES (?, ?, ?)";
String sqlDetalle = "INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio_unitario) VALUES (?, ?, ?, ?)";

try {
    con.setAutoCommit(false);

    PreparedStatement psVenta = con.prepareStatement(string:sqlVenta, Statement.RETURN_GENERATED_KEYS);
    psVenta.setInt(1, pedidoResultado.getIdCliente());
    psVenta.setDouble(2, pedidoResultado.calcularTotalFinal());
    psVenta.setString(3, pedidoResultado.getFecha());
    psVenta.executeUpdate();

    // Obtener el ID asignado a la venta actual
    ResultSet rsKeys = psVenta.getGeneratedKeys();
    int idVentaGenerado = 0;
    if (rsKeys.next()) {
        idVentaGenerado = rsKeys.getInt(1);
    }

    PreparedStatement psDetalle = con.prepareStatement(string:sqlDetalle);
    for (int i = 0; i < pedidoResultado.getProductos().size(); i++) {
        psDetalle.setInt(1, idVentaGenerado);
        psDetalle.setInt(2, pedidoResultado.getIdProductos().get(i));
        psDetalle.setInt(3, pedidoResultado.getCantidades().get(i));
        psDetalle.setDouble(4, pedidoResultado.getPrecios().get(i));
        psDetalle.addBatch(); // Procesamiento por lotes
    }
    psDetalle.executeBatch();

    con.commit();
    JOptionPane.showMessageDialog(parentComponent: this, message: "Pedido registrado con éxito en la Base de Datos!");
}

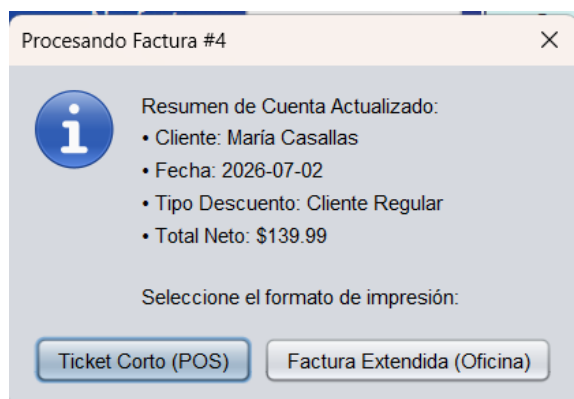
```

Statement.RETURN_GENERATED_KEYS: Configura la sentencia INSERT para que devuelva la clave primaria (id_venta) recién creada por el motor de la base de datos (AUTO_INCREMENT).

Encapsulamiento de Datos: Se extraen los valores directamente del objeto pedidoResultado mediante métodos *getter* (getIdCliente(), calcularTotalFinal(), getFecha()), respetando los principios de Programación Orientada a Objetos.

getGeneratedKeys(): Recupera la llave autograbada y la asigna a la variable idVentaGenerado, necesaria para enlazar los productos del detalle con esta venta específica.

Por último, el botón "Factura" permite consultar y reimprimir facturas previamente almacenadas en la base de datos. Esta funcionalidad facilita la realización de ajustes o correcciones en la información registrada cuando sea necesario. Además, ofrece la posibilidad de generar e imprimir facturas tanto para personas naturales como para personas jurídicas, adaptándose a los requerimientos de cada tipo de cliente.



```
private void b_facturaActionPerformed(java.awt.event.ActionEvent evt) {
    int filaSeleccionada = tabla1.getSelectedRow();

    if (filaSeleccionada == -1) {
        JOptionPane.showMessageDialog(parentComponent, this, message: "Por favor, seleccione una factura de la tabla."
        return;
    }

    clases.DatosFactura datosFactura = new clases.DatosFactura(
        idFactura:tabla1.getValueAt(row: filaSeleccionada, column:0).toString(),
        cliente: tabla1.getValueAt(row: filaSeleccionada, column:1).toString(),
        total: tabla1.getValueAt(row: filaSeleccionada, column:5).toString(),
        fecha: tabla1.getValueAt(row: filaSeleccionada, column:6).toString()
    );

    int respuestaEdicion = JOptionPane.showConfirmDialog(parentComponent: this,
        message: "¿Desea modificar los datos del cliente o fecha antes de imprimir?",
        title: "Actualizar Datos de Facturación", optionType: JOptionPane.YES_NO_OPTION);
}
```

Imagen codigo imprimir factura

Este evento gestiona el inicio del proceso de emisión o impresión de una factura a partir de un registro seleccionado en la interfaz gráfica. Sus tareas principales son **validar la selección del usuario, encapsular los datos de la fila elegida** en un objeto de modelo (DatosFactura) y **consultar al usuario si requiere modificar la información** (cliente o fecha) antes de proceder con el documento final.

En el encapsulamiento de datos tenemos **new clases.DatosFactura(...)**: Se crea una instancia del objeto DatosFactura pasando como parámetros las celdas de la fila seleccionada mediante `tabla1.getValueAt(...)`:

- **Columna 0:** Identificador o Folio de la factura.
- **Columna 1:** Nombre / ID del Cliente.
- **Columna 5:** Fecha de emisión.
- **Columna 6:** Monto / Total.
- **Uso del patrón DTO / Encapsulamiento:** Extraer los datos de la tabla gráfica y empaquetarlos en un objeto dedicado facilita el transporte seguro de la información entre la interfaz y el módulo de impresión o edición.

```

double total = Double.parseDouble(s: datosFactura.getTotal());
clases.Descuento descuentoAplicado;
if (total >= 50000) {
    descuentoAplicado = new clases.DescuentoMayorista(totalOriginal: total);
} else {
    descuentoAplicado = new clases.DescuentoRegular(totalOriginal: total);
}

double totalFinal = descuentoAplicado.calcularTotalConDescuento();

String[] opciones = {"Ticket Corto (POS)", "Factura Extendida (Oficina)"};
int seleccion = JOptionPane.showOptionDialog(parentComponent: this,
    "Resumen de Cuenta Actualizado:\n" +
    "• Cliente: " + datosFactura.getCliente() + "\n" +
    "• Fecha: " + datosFactura.getFecha() + "\n" +
    "• Tipo Descuento: " + descuentoAplicado.getTipo() + "\n" +
    "• Total Neto: $" + totalFinal + "\n\n" +
    "Seleccione el formato de impresión:",
    "Procesando Factura #" + datosFactura.getIdFactura(),
    JOptionPane.DEFAULT_OPTION,
    JOptionPane.INFORMATION_MESSAGE,
    icon: null, options: opciones, opciones[0]);

if (seleccion == -1) return;

Imprimible documento;
if (seleccion == 0) {
    documento = new FacturaTicket(idFactura: datosFactura.getIdFactura(), cliente:
} else {
    documento = new FacturaOficina(idFactura: datosFactura.getIdFactura(), cliente:
)

```

Imagen código aplicación descuento

Este fragmento concluye el flujo de facturación ejecutando tres tareas clave:

1. **Evaluación de descuentos:** Determina de forma dinámica qué tipo de descuento corresponde según el monto total.
2. **Selección del formato de salida:** Permite al usuario elegir mediante la interfaz el formato del comprobante (*Ticket POS* o *Factura de Oficina*).
3. **Generación del documento:** Utiliza una interfaz (*Imprimible*) para procesar e imprimir el documento seleccionado sin importar su tipo específico (Polimorfismo).

Aplicación de Lógica de Negocio y Polimorfismo (Descuento)

- **Double.parseDouble(...):** Convierte el monto plano (texto) del objeto datosFactura a un tipo numérico para evaluar las reglas del negocio.
- **Evaluación Condicional:**
 - Si el total es mayor o igual a \$50,000, instancia la clase DescuentoMayorista.
 - De lo contrario, instancia la clase DescuentoRegular.
- **Polimorfismo:** Ambas clases heredan de la clase base Descuento. Esto permite calcular el total llamando al método calcularTotalConDescuento(), el cual aplicará el porcentaje de descuento correspondiente de manera automática según el tipo de objeto creado.

Interfaz de Usuario y Diálogo de Opciones (JOptionPane.showOptionDialog)

- Muestra un cuadro de diálogo dinámico que presenta un **Resumen de la Cuenta** (Cliente, Fecha, Tipo de Descuento aplicado y Total Neto final).
- Ofrece dos botones de decisión mediante el arreglo opciones:
 1. **Opción 0:** Ticket Corto (POS).
 2. **Opción 1:** Factura Extendida (Oficina).

- **if (seleccion == -1) return;;** Si el usuario cierra la ventana de opciones sin seleccionar nada, cancela el proceso evitando un comportamiento inesperado.

Uso de Interfaces o Clases Abstractas (Imprimible)

- **Declaración del Polimorfismo (Imprimible documento):** La variable documento se declara como tipo Imprimible (una interfaz o clase abstracta).
- **Instanciación Dinámica:**
 - Si eligió la opción 0, se crea una instancia de FacturaTicket.
 - Si eligió la opción 1, se crea una instancia de FacturaOficina.
- **Invocación Unificada (documento.generarDocumento()):** No importa cuál de las dos facturas se haya instanciado, ambas garantizan tener el método generarDocumento(). Esto permite enviar la orden de impresión sin acoplar el código a un formato específico.

Conexión a base de datos MySQL.

El fragmento de código define la clase Conexiones dentro del paquete conexiones. Su objetivo principal es gestionar la conexión entre la aplicación en Java y un motor de base de datos MySQL local (producción) mediante la API JDBC (*Java Database Connectivity*).

A través de esta conexión, la aplicación en Java ejecuta las consultas a la base de datos para alimentar la interfaz gráfica. Esto proporciona un entorno dinámico que permite al usuario ajustar la disposición de las columnas y visualizar en tiempo real las actualizaciones en las tablas del sistema de gestión de la ferretería

Bloque try-catch:

- En el bloque try, invoca DriverManager.getConnection(url, usuario, pass). Este método valida las credenciales y el estado del servidor. Si tiene éxito, devuelve el objeto de conexión y muestra un mensaje explicativo en la consola.
- Si ocurre un fallo (ej. servicio de MySQL apagado, contraseña incorrecta o base de datos inexistente), el bloque catch captura la excepción (Exception e) e imprime el mensaje de error correspondiente sin detener abruptamente la aplicación.

```

1 package conexiones;
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.SQLException;
5
6 public class Conexiones {
7     private static final String url = "jdbc:mysql://localhost:3306/produccion";
8     private static final String usuario = "root";
9     private static final String pass = "admin";
10
11     public static Connection conectar() {
12         Connection conexion = null;
13         try {
14             conexion = DriverManager.getConnection(url, usuario, pass);
15             System.out.println("Conexion correcta a Mysql");
16         } catch (Exception e) {
17             System.out.println("Error de Conexion " + e);
18         }
19         return conexion;
20     }
21
22     public static void main(String[] args) {
23         Conexiones test = new Conexiones();
24         test.conectar();
25     }
26 }

```

3. Diseño Base Datos Cliente

Para el desarrollo del sistema se utilizó una base de datos implementada en MySQL, la cual permite almacenar y gestionar la información de manera eficiente en el equipo del cliente. Esta solución ofrece la capacidad de manejar volúmenes considerables de datos, garantizando un acceso rápido organizado y seguro a la información relacionada con clientes, empleados, inventario y pedidos. El sistema utiliza MySQL como sistema gestor de bases de datos relacionales debido a su estabilidad y soporte para transacciones e integridad referencial (Oracle, 2025)

3.1 Tablas de datos

Siguiendo el objetivo principal del sistema, se busca facilitar la gestión y administración de la información de la ferretería para ello el sistema permitirá administrar los datos de los clientes, controlar el acceso de los empleados según sus diferentes cargos y niveles de responsabilidad, gestionar las existencias en el almacén y procesar los pedidos de manera eficiente, teniendo en cuenta la disponibilidad de productos y la información registrada de clientes y empleados. De esta forma, se optimizan los procesos operativos y se mejora la organización general de la empresa.

La tabla **usuarios** nos permitirá saber el cargo de los empleados y a la vez asignarles un usuario y contraseña para poder ingresar al sistema de administración de la Ferretería

id_usuario	nombre_usuario	clave	nombre	apellido	departamento	puesto
1	jperis	clave123	Juan	Peris	Ventas	Asesor Comercial
2	mlopez	admin2026	María	López	Administración	Gerente General
3	cmendoza	segura#1	Carlos	Mendoza	Almacén	Supervisor de Stock
4	agomez	gomez99	Ana	Gómez	Ventas	Cajera Principal
5	lmartinez	luis_mtz	Luis	Martínez	Sistemas	Soporte Técnico
6	erodriguez	elena12	Elena	Rodríguez	Contabilidad	Contadora
7	psanchez	pedroX	Pedro	Sánchez	Almacén	Auxiliar de Bodega
8	scastro	sonia_c	Sonia	Castro	Recursos Humanos	Analista de personal
9	jvilla	jorgeV	Jorge	Villa	Ventas	Asesor Comercial
10	lfernandez	lucia_f	Lucía	Fernández	Marketing	Diseñadora
11	rrey	ricardoR	Ricardo	Rey	Compras	Encargado de Com...
12	msofia	marta123	Marta	Sofía	Atención al Cliente	Recepcionista
13	dtorres	diegoT	Diego	Torres	Almacén	Montacarguista
14	parce	patriciaA	Patricia	Arce	Ventas	Cajera Turno Tarde
15	asilva	andresS	Andrés	Silva	Seguridad	Supervisor de Segu...

La tabla **Clientes** almacena la información personal y de contacto de los clientes del negocio, permitiendo mantener un registro organizado y actualizado de cada uno de ellos. En esta tabla se guardan datos necesarios para identificarlos y gestionar adecuadamente las transacciones comerciales realizadas.

Además, la información registrada permite consultar el historial de compras de cada cliente, facilitando el seguimiento de sus transacciones dentro del negocio. Asimismo, proporciona los datos requeridos para la generación e impresión de facturas, asegurando que cada documento contenga la información correcta del cliente y cumpla con los requisitos establecidos para el proceso de facturación.

id_cliente	nombre	direccion	telefono
1	Juan Pérez	Av. Siempre Viva 742	555-1234
2	María Casallas	av cali 2568	555-5678
3	Carlos Mendoza	Diagonal 45 #12-34	555-9012
4	Ana Gómez	Av. Central 890	555-3456
5	Luis Martínez	Calle 10 #5-20	555-7890
6	Elena Rodríguez	Carrera 15 #78-90	555-2345
7	Pedro Sánchez	Calle del Sol 45	555-6789
8	Sonia Castro	Av. de la Luna 12	555-0123
9	Jorge Villa	Paseo de los Héroes 99	555-4567

La tabla **Productos** almacena la información de cada artículo disponible en el negocio, incluyendo datos importantes como su nombre, precio, descripción y cantidad en existencia. Gracias a esta tabla, es posible consultar y controlar el stock disponible en el almacén, permitiendo una gestión eficiente del inventario y evitando la venta de productos sin disponibilidad.

Además, cada producto se encuentra asociado a una categoría mediante el campo **id_categoria**, lo que permite identificar la familia o grupo al que pertenece. Esta relación facilita la organización de los productos, mejora las búsquedas dentro del sistema y permite generar reportes más estructurados según las diferentes categorías existentes.

id_producto	nombre	descripcion	precio	existencia	id_categoria	fecha_ingreso
1	Smartphone X	Teléfono inteligente de última generación	799.99	80	1	2026-01-10
2	Camiseta de Algodón	Camiseta cómoda talla M	19.99	150	2	2026-02-15
3	Cafetera Express	Cafetera automática de acero inoxidable	120.00	20	3	2026-03-01
4	Novela de Ficción	Libro de tapa dura, 400 páginas	15.50	100	4	2026-01-20
5	Balón de Fútbol	Balón reglamentario tamaño 5	25.00	80	5	2026-02-10
6	Bloques de Construcción	Juguete didáctico de 100 piezas	30.00	45	6	2026-03-12
7	Aceite para Motor	Aceite sintético 5W-30	45.00	35	7	2026-04-05
8	Crema Hidratante	Para todo tipo de piel 200ml	12.99	80	8	2026-04-18
9	Alimento para Perro	Sabor carne y vegetales 10kg	35.00	40	9	2026-05-02
10	Juego de Destornilladores	Kit de 12 piezas imantadas	18.50	85	10	2026-05-10
11	Tenis Deportivos	Zapatillas para correr talla 40	85.00	30	11	2026-05-25
12	Manguera de Jardín	Manguera expandible de 15 metros	22.00	15	12	2026-06-01
13	Silla de Oficina	Silla ergonómica con soporte lumbar	140.00	12	13	2026-06-14
14	Vitaminas C 1000mg	Suplemento en tabletas x100	9.99	200	14	2026-06-20
15	Café Gourmet en Grano	Bolsa de café de origen 1kg	14.50	70	15	2026-07-01

La tabla **Detalle_Venta** tiene como propósito registrar los productos asociados a cada venta realizada, estableciendo una relación entre la transacción y el cliente correspondiente. En ella se almacena información como los productos vendidos, las cantidades, los precios unitarios y los subtotales de cada artículo.

Además, esta tabla permite actualizar automáticamente las existencias del almacén a medida que se realizan las ventas, garantizando que el inventario refleje la disponibilidad real de los productos. Asimismo, proporciona la información necesaria para calcular el valor total de la compra y generar la factura correspondiente para el cliente, asegurando un registro preciso y organizado de cada transacción comercial.

id_detalle_venta	id_venta	id_producto	cantidad	precio_unitario
1	1	1	1	799.99
2	1	5	1	25.00
3	2	2	1	19.99
4	2	3	1	120.00
5	3	4	1	15.50
6	4	5	2	25.00
7	5	7	1	45.00
8	6	8	1	12.99
9	7	9	1	35.00

La tabla **Venta** almacena la información general de cada transacción realizada por un cliente, incluyendo datos como la fecha de la venta, el cliente asociado y el valor total de la compra. Esta tabla permite llevar un control histórico de las ventas efectuadas y facilita la consulta de la información comercial de la empresa.

Además, se relaciona con la tabla **Detalle_Venta**, donde se registran los productos vendidos, las cantidades y los precios correspondientes. Esta relación permite consolidar toda la información de la transacción y generar de manera precisa la factura de compra del cliente, mostrando el detalle de los productos adquiridos y el total a pagar.

id_venta	id_cliente	fecha_venta	total_venta
1	1	2026-07-01	824.99
2	2	2026-07-02	139.99
3	3	2026-07-02	15.50
4	4	2026-07-03	50.00
5	5	2026-07-03	45.00
6	6	2026-07-04	12.99
7	7	2026-07-04	35.00
8	8	2026-07-05	18.50

La tabla **Categorías** tiene como finalidad clasificar los productos según la familia o grupo al que pertenecen, permitiendo una gestión más organizada y eficiente de la información. Gracias a esta clasificación, los productos no se manejan como elementos individuales aislados, sino como parte de conjuntos con características similares.

Esta estructura facilita la búsqueda, consulta y administración del inventario, además de simplificar tareas como el registro de nuevos productos, la generación de reportes y el control de existencias. De esta manera, el sistema mantiene una organización más clara y escalable, mejorando la gestión de los productos dentro de la empresa.

id_categoria	nombre_categoria
1	Electrónica
2	Ropa
3	Hogar y Cocina
4	Libros
5	Deportes
6	Juguetes
7	Automotriz
8	Belleza

Por último, la tabla **Almacén** permite llevar el control de las existencias disponibles de los productos dentro del negocio. En ella se registran los movimientos de inventario,

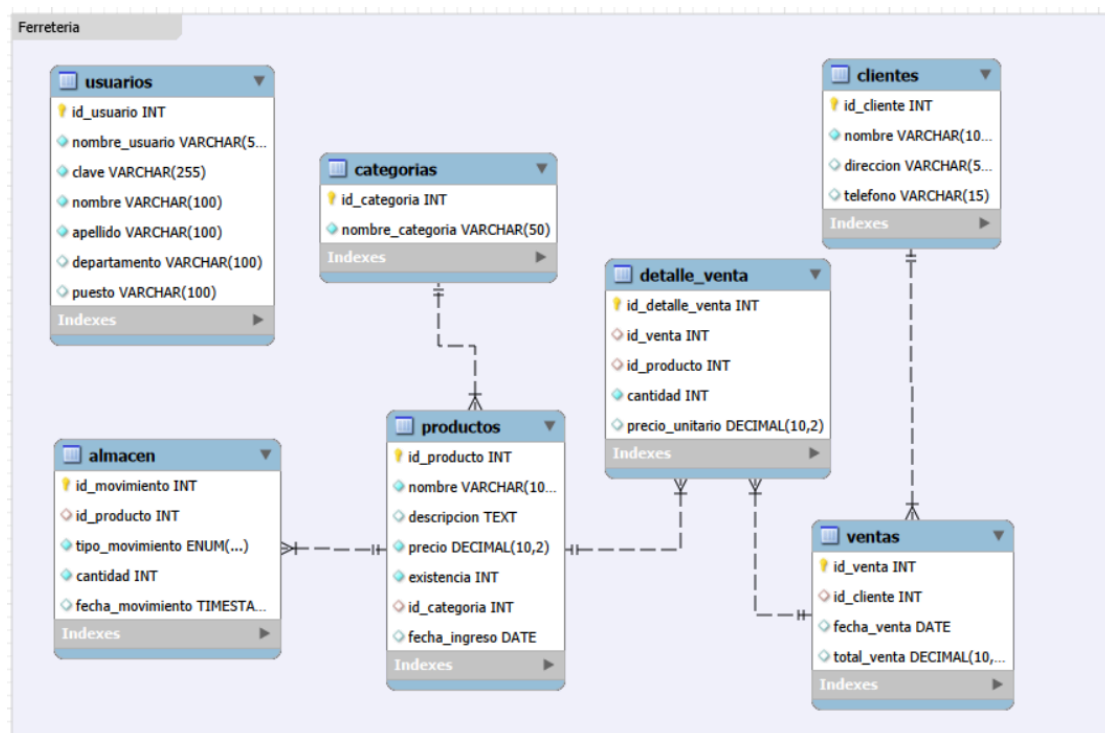
especialmente los ingresos de nuevo stock, garantizando que la información se mantenga actualizada y refleje la cantidad real de productos disponibles.

Esta tabla facilita la gestión y supervisión del inventario, permitiendo registrar reposiciones de mercancía, controlar la disponibilidad de los productos y apoyar la toma de decisiones relacionadas con las compras y ventas. Además, contribuye a evitar faltantes o excesos de inventario, asegurando un mejor control de los recursos almacenados.

id_movimiento	id_producto	tipo_movimiento	cantidad	fecha_movimiento
1	1	Entrada	10	2026-07-10 10:36:03
2	2	Salida	5	2026-07-10 10:36:03
3	3	Entrada	15	2026-07-10 10:36:03
4	4	Salida	2	2026-07-10 10:36:03
5	5	Entrada	20	2026-07-10 10:36:03
6	6	Salida	8	2026-07-10 10:36:03
7	7	Entrada	12	2026-07-10 10:36:03
8	8	Salida	4	2026-07-10 10:36:03
9	9	Entrada	30	2026-07-10 10:36:03
10	10	Salida	1	2026-07-10 10:36:03
11	11	Entrada	25	2026-07-10 10:36:03
12	12	Salida	3	2026-07-10 10:36:03
13	13	Entrada	5	2026-07-10 10:36:03

3.2 Modelo relacional de tablas.

A continuación, se presenta la estructura de la base de datos diseñada para garantizar una correcta organización y gestión de la información. Esta estructura permite mantener la integridad de los datos, evitar duplicidades y prevenir cruces de información entre tablas o módulos que no estén relacionados, asegurando así un almacenamiento eficiente, seguro y coherente dentro del sistema.



CAPÍTULO IV. RESULTADOS

4.1 Resultados obtenidos

El desarrollo del proyecto permitió cumplir el objetivo general de implementar una aplicación de escritorio para la administración de una ferretería utilizando Java Swing, Programación Orientada a Objetos y una base de datos MySQL. La aplicación integra una interfaz gráfica que facilita la gestión de la información y centraliza las principales operaciones del negocio.

Con respecto a los objetivos específicos, se obtuvieron los siguientes resultados:

Se implementó un módulo de clientes que permite registrar, consultar, actualizar y eliminar la información almacenada en la base de datos, facilitando la administración de los datos de los clientes.

Se desarrolló el módulo de productos, mediante el cual es posible administrar el inventario de la ferretería, registrando nuevos productos, modificando su información y eliminando registros cuando es necesario.

Se implementó el módulo de ventas, el cual permite generar pedidos locales y a domicilio, calcular automáticamente los valores de la compra, aplicar descuentos según el tipo de cliente y generar los documentos correspondientes a cada venta.

Se desarrolló un sistema de autenticación mediante un inicio de sesión, el cual valida las credenciales de los usuarios antes de permitir el acceso a los diferentes módulos de la aplicación.

Se implementó la generación de documentos de venta mediante las clases FacturaTicket y FacturaOficina, permitiendo producir diferentes formatos de factura de acuerdo con las necesidades del sistema.

Además del cumplimiento de los objetivos funcionales, durante el desarrollo del proyecto se aplicaron los principios de la Programación Orientada a Objetos mediante el uso de clases abstractas, herencia, encapsulamiento, polimorfismo e interfaces, logrando una estructura de código organizada, reutilizable y fácil de mantener.

En general, los resultados obtenidos demuestran que el sistema cumple con los requerimientos establecidos para el proyecto, proporcionando una solución funcional para la administración de una ferretería y reemplazando el manejo manual de la información por un sistema centralizado y conectado a una base de datos.

4.2 Problemas encontrados durante el desarrollo

Durante el desarrollo del proyecto se presentaron diversas dificultades técnicas inicialmente se produjeron errores al establecer la conexión entre Java y MySQL debido a la configuración del controlador JDBC y las credenciales de acceso

Posteriormente fue necesario reorganizar la estructura de las clases para implementar correctamente la herencia y el polimorfismo mediante clases abstractas e interfaces.

Finalmente, la sincronización entre la interfaz gráfica y la base de datos requirió validar cuidadosamente las operaciones CRUD para garantizar la consistencia de la información almacenada.

Cada uno de estos problemas fue solucionado mediante pruebas sucesivas y refactorización del código, obteniendo un sistema estable y funcional.

Conclusiones

El sistema de ventas fue diseñado para que el proceso de facturación no dependa de una única implementación. Mediante las clases `PedidoLocal` y `PedidoDomicilio` se logró que el cálculo del valor final de una compra cambiara automáticamente según el tipo de pedido, permitiendo incorporar el cargo por envío únicamente cuando corresponde. Esta estructura evita duplicar código y facilita la incorporación de nuevas modalidades de venta sin modificar la lógica existente.

La implementación de la jerarquía `Descuento`, `DescuentoMayorista` y `DescuentoRegular` permitió separar completamente las reglas comerciales del resto del sistema. Gracias a ello, el proceso de facturación únicamente solicita el cálculo del descuento, mientras que cada clase decide internamente cómo aplicar la promoción correspondiente. Esto demuestra que la lógica del negocio puede modificarse sin afectar los módulos de ventas ni la interfaz gráfica.

La generación de documentos fue desarrollada utilizando la clase abstracta `DocumentoFactura` y las clases `FacturaTicket` y `FacturaOficina`, permitiendo producir dos formatos de comprobante a partir de la misma información almacenada en la base de datos. Además, la utilización de la interfaz `Imprimible`

permitió que cualquier documento pudiera imprimirse mediante una única llamada al método `generarDocumento()`, independientemente del formato seleccionado por el usuario.

Durante el registro de una venta se implementó un proceso transaccional en MySQL donde primero se almacena la cabecera de la venta, posteriormente se recupera el identificador generado automáticamente (`id_venta`) y finalmente se registran todos los productos asociados en la tabla `detalle_venta`. Este procedimiento garantiza la relación correcta entre ambas tablas y evita inconsistencias cuando una operación no puede completarse correctamente.

El diseño de la base de datos permitió mantener la integridad de la información mediante relaciones entre las tablas de clientes, productos, categorías, ventas, detalle de ventas, almacén y usuarios. Asimismo, el uso de consultas preparadas (`PreparedStatement`) en todos los módulos redujo el riesgo de errores durante la captura de información y fortaleció la seguridad del sistema frente a intentos de inyección SQL.

En conjunto, el proyecto no solo permitió administrar clientes, productos y ventas, sino que integró mecanismos de autenticación, control de inventario, generación de facturas, aplicación automática de descuentos y actualización de existencias en una única aplicación de escritorio. Esto demuestra que los principios de la Programación Orientada a Objetos fueron aplicados para resolver problemas específicos del funcionamiento de una ferretería y no únicamente como parte de un ejercicio académico.

Referencias Bibliográficas

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). **The Unified Modeling Language User Guide** (2nd ed.). Addison-Wesley Professional.

Oracle. (2025). **Java Database Connectivity (JDBC) Tutorial**.
<https://docs.oracle.com/javase/tutorial/jdbc/>

Oracle. (2025). **Creating a GUI With Swing**.
<https://docs.oracle.com/javase/tutorial/uiswing/>

Oracle. (2025). **MySQL 8.0 Reference Manual**.
<https://dev.mysql.com/doc/refman/8.0/en/>

